Redis Cluster: Data Sharding and High Availability

ID: DOC-DB-316	Category: MongoDB, PostgreSQL and Redis	Version: 1.5

1. Algorithmic Sharding via Hash Slots

Redis Cluster provides a way to run a Redis installation where data is automatically partitioned (sharded) across multiple Redis nodes. Unlike consistent hashing algorithms used by some other distributed databases, Redis Cluster uses a deterministic concept called Hash Slots. There are exactly 16,384 hash slots in a Redis Cluster.

To calculate which hash slot a particular key belongs to, Redis uses the CRC16 checksum of the key, modulo 16384. Every master node in the cluster is responsible for a contiguous subset of these 16,384 slots. When a client requests a key, it connects to a node; if the node holds the slot for that key, it serves the request. If not, it replies with a `-MOVED` redirection error, pointing the client to the correct node.

2. High Availability and Replica Nodes

To ensure high availability in the event of hardware failures or network partitions, Redis Cluster utilizes a primary-replica (master-slave) architecture. Every master node should have at least one replica node. The replica asynchronously replicates the master's dataset.

Nodes continuously ping each other via a binary Gossip Protocol on a dedicated cluster bus port (usually the client port + 10000). If a majority of master nodes determine that a specific master is unreachable, an automatic failover is triggered. The cluster promotes one of the failing master's replicas to become the new master, assigns the corresponding hash slots to it, and the cluster continues to operate without manual intervention.

3. Limitations on Multi-Key Operations

A fundamental restriction of Redis Cluster is that multi-key operations (like `MGET`, `MSET`, or transactions via `MULTI/EXEC`) are only supported if all keys involved in the operation map to the exact same hash slot. Attempting a cross-slot operation will result in a `CROSSSLOT` error.

To bypass this limitation, developers must use Hash Tags. By enclosing a substring of the key in curly braces `{}`, Redis will only hash the contents inside the braces. For example, the keys `user:{123}:profile` and `user:{123}:settings` will guaranteed map to the same hash slot because only the string `123` is evaluated by the CRC16 algorithm. This allows developers to co-locate related data on the same physical node to support complex multi-key transactions.

4. Client Compatibility

Not all Redis clients support cluster mode natively. A cluster-aware client must maintain an internal mapping of hash slots to node IP addresses. Instead of blindly sending commands and handling `-MOVED` redirects (which adds network latency), a smart client fetches the `CLUSTER SLOTS` topography on startup and routes queries directly to the appropriate master, falling back to redirects only when the cluster topology changes dynamically.